

RFE-Based Feature Selection for Multiclass Intrusion Detection: Performance Comparison of MLP, Random Forest, Decision Tree, and XGBoost

Aryam Mansour
Reem Sukkari
Razan Hamchou
Rawdha Abdelaziz

Abstract- As cyber threats continue to evolve, accurate and efficient intrusion detection has become increasingly important for securing computer networks. This study applies Recursive Feature Elimination (RFE) for feature selection on the UNSW-NB15 dataset and evaluates four machine learning classifiers-Multi-Layer Perceptron (MLP), Random Forest, Decision Tree, and XGBoost- for multiclass intrusion detection. The models are compared using accuracy, precision, recall, and F1-score to assess their effectiveness in detecting and classifying network attacks. The results provide insight into the performance of tree-based, ensemble, and neural network approaches when combined with RFE for multiclass intrusion detection.

I. INTRODUCTION

In parallel with the increased development of digital communication, network attacks have become much more common, diverse, and sophisticated. Modern computer networks face a variety of security threats, such as denial-of-service attacks, exploitation, network scanning, and many others that can compromise the availability, integrity, and confidentiality of data. Therefore, the issue of correctly recognizing the type of network traffic and distinguishing it from various malicious activities is becoming more relevant in the field of cybersecurity.

Traditional security mechanisms often rely on predefined rules or known attack signatures, which may be insufficient when dealing with new, complex, or large-scale attack behaviors. For these reasons, there is a need to develop and apply more sophisticated tools that are adaptive and intelligent and can analyze network traffic and detect suspicious behavior automatically. The classification of cyberattacks is one of these tools that allow not only the detection of malicious behavior but also its correct classification. Such classification can support faster response, better threat analysis, and more effective intrusion detection systems.

This project will examine the problem of multiclass cyberattack classification, which involves applying machine learning algorithms to recognize several categories of network attacks and distinguish them from the normal traffic. The goal is to evaluate how effectively different machine learning models can recognize and classify these attack types, and to compare their performance in handling a realistic cybersecurity classification problem.

Figure. 1 summarizes the overall workflow followed in this study, from initial data preprocessing through model training and evaluation.

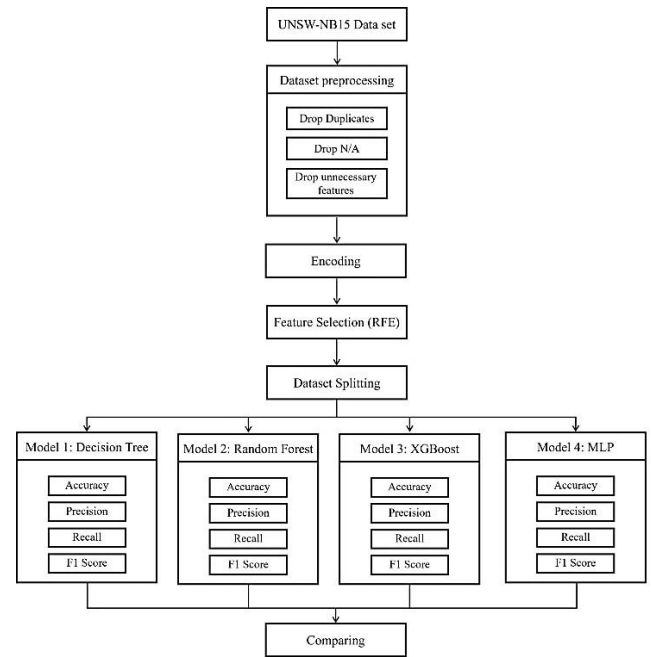


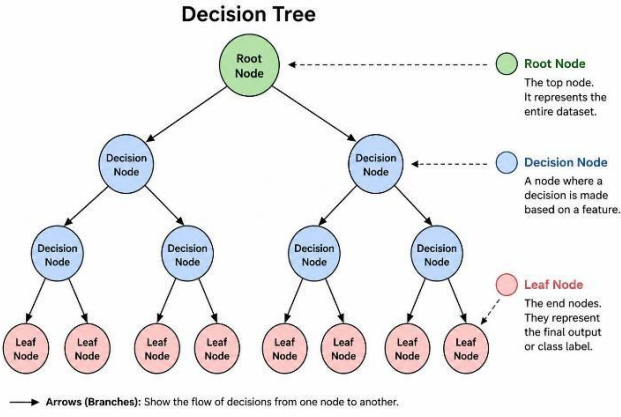
Figure 1

II. TECHNICAL BACKGROUND

A. Decision tree classifier

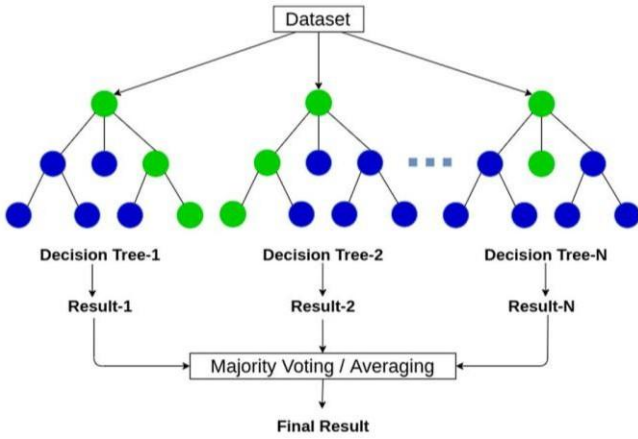
A Decision Tree is a supervised machine learning algorithm that performs classification using a tree-like model of decisions and their possible consequences. The tree contains internal nodes representing decision rules; the branches represent the outcomes of these decisions, and the leaf nodes represent the classes to which the sample belongs. In this particular project, the Decision Tree model is applied to classify the network traffic records according to their cyberattack category.

This model learns from the training data and identifies the features splits that are most useful for distinguishing between different classes. As a result, the model is suitable for performing multiclass classification tasks such as the one presented in the paper. One of the key strengths of the Decision Tree algorithm is its ease of understanding and interpretation. In addition, the classification algorithm itself can be represented as a sequence of decision rules, which makes the model transparent and comprehensible for users. Another advantage of this algorithm is its effectiveness in working with tabular data.



B. Random Forest

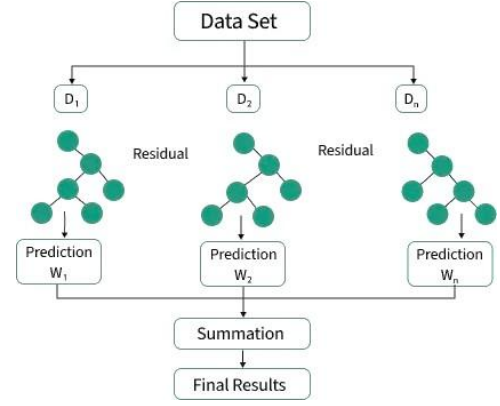
Random Forest is an ensemble learning method that builds a large collection of decision trees and combines their predictions through **majority voting** (for classification). The trees in Random Forest are built in **parallel** it doesn't wait for a tree to finish to start with the other. Each tree is trained on a bootstrapped sample of the training data (**bagging**), and at each split only a random subset of features is considered. This double randomization decorrelates the individual trees, reducing the variance of the overall model relative to a single decision tree and improving generalization.



C. XGBoost

XGBoost is an ensemble learning method that builds decision trees sequentially, with each new tree trained to correct the residual errors of the tree before it. It optimizes a loss function with gradient descent, allowing predictions to improve iteratively. To control overfitting, it applies L1 and L2 regularization, and it employs histogram based split finding to improve computational efficiency on large datasets.

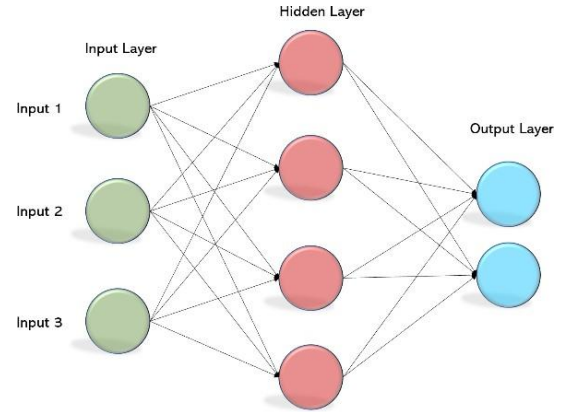
Beyond its core structure, XGBoost incorporates several mechanisms to further improve robustness and efficiency. Shrinkage, controlled by a learning rate, scales down each tree's contribution to the overall model, balancing training speed against generalization ability. Subsampling of rows and columns, similar to the approach used in random forests, reduces correlation between individual trees. Additionally, early stopping allows training to halt once performance on a validation set stops improving. Together, these techniques serve as safeguards against overfitting.



C. MLP

Multi-Layer Perceptron is a supervised feedforward artificial neural network that consists of an input layer, one or more hidden layers, and an output layer. The network consists of neurons connected by weighted links that transmit signals between neurons. The input signals are propagated in a forward direction through the network, while the weights are adjusted using the **backpropagation** learning algorithm to reduce the difference between the predicted and desired outputs.

The hidden layers enable the MLP to learn complex and non-linear relationships between the input features and output classes, making it suitable for multiclass classification tasks such as the one presented in this paper. During training, the network repeatedly updates its weights until the prediction error is minimized, allowing it to achieve high classification performance on complex datasets.



III. DATASET PREPROCESSING AND FEATURE SELECTION

A. Dataset Description and Preparation

This project uses the **UNSW-NB15** dataset for intrusion detection. The dataset was developed by the Australian Centre for Cyber Security (ACCS) at the University of New South Wales (UNSW) as a modern benchmark for evaluating intrusion detection systems. It was created by generating realistic normal network traffic and various cyberattacks using the IXIA PerfectStorm traffic generator in the UNSW Cyber Range Lab.

The complete dataset contains more than 2.5 million network flow records. To reduce computational complexity while maintaining representative data, this project uses the widely adopted **257,673 observation sample**. Each record contains **45 extracted features** that represent the characteristics of a network flow.

The dataset provides two target labels. The **‘label’** feature serves as a binary classification target, distinguishing between normal and malicious traffic, making the dataset suitable for binary intrusion detection tasks. The **‘attack_cat’** feature, on the other hand, is a multiclass label identifying the specific type of attack present. This project uses attack_cat to classify malicious traffic into one of **nine** categories: **Fuzzers, Analysis, Backdoors, DoS, Exploits, Generic, Reconnaissance, Shellcode, and Worms**.

The following Python libraries were imported to support data preprocessing, feature selection, model training, and performance evaluation.

Module/Library	Function	Purpose
pandas as pd	read_csv() dropna()	Data preprocessing
sklearn.feature_selection	RFE	Feature selection
sklearn.preprocessing	LabelEncoder StandardScaler	Encode categorical features and normalize numerical features
sklearn.model_selection	train_test_split()	Holdout dataset splitting
sklearn	tree	Decision tree model
sklearn.neural_network	MLPClassifier	MLP model
sklearn.ensemble	RandomForestClassifier	Random Forest Model
xgboost	XGBClassifier	XGBoost Model
sklearn.metrics	accuracy_score() precision_score() recall_score() f1_score()	Performance Evaluation

Although the official dataset has predefined training and testing sets, these sets exhibit differences in class distributions that may introduce **bias** during model evaluation. To obtain a more randomized split, the original training and testing files were first **concatenated** into a single dataset and loaded to **Google Colab**. The combined dataset will then be randomly split into new training and testing sets after preprocessing and feature selection.

1 – Loading the dataset

```
from google.colab import files
uploaded = files.upload()
df = pd.read_csv("intrusion.csv")
```

B. Data Cleaning

After inspecting the dataset, we found out that the ‘dur’ had small numbers stored as text which would’ve negatively impacted our models if left untreated. We forced this feature to float in code.

```
df['dur'] = pd.to_numeric(df['dur'], errors='coerce')
```

Duplicate records and rows with missing values were removed. The placeholder “-”, used in categorical fields to denote an unspecified value, was treated as missing and the corresponding rows dropped.

```
df.dropna(inplace = True)

df = df.replace('-', pd.NA)
df.dropna(inplace=True)
```

The id and binary label columns were removed, since this study targets multiclass classification via attack_cat.

```
df.drop(columns=['id', 'label'], inplace=True)
```

After cleaning, the dataset was reduced from 257,673 to **116,352 records**. A side effect of dropping “-” placeholder rows is that every **Shellcode** record was removed, since all such records contained this placeholder in the service field, therefore, our models distinguish between 8 attack types instead of 9, which is noted as a limitation.

Category	Number of records
Generic	57956
Normal	29113
Exploits	21480
DoS	2508
Fuzzers	2266
Reconnaissance	2207
Analysis	564
Worms	148
Backdoor	110

As the distribution shows, the dataset exhibits a significant **class imbalance**, with some attack categories containing considerably fewer samples than others. This may cause our AI models to become biased toward majority classes and reduce their ability to detect minority attack types, and this is noted as another limitation.

C. Categorical Encoding

The categorical features **proto**, **service**, and **state**, along with the target variable **attack_cat**, were transformed using **Label Encoding** to convert categorical features into numerical representations that can be processed by our AI models. Since most machine learning models require numerical input, encoding enables the use of these categorical features during model training.

```
le = LabelEncoder()
df["proto"] = le.fit_transform(df["proto"])
df["service"] = le.fit_transform(df["service"])
df["state"] = le.fit_transform(df["state"])
df["attack_cat"] = le.fit_transform(df["attack_cat"])
```

For the MLP, feature scaling (Section V) was additionally applied to reduce the impact of differences in feature ranges and improve model training stability.

D. Feature Selection

Before feature selection, we divided the dataset into input (x) and output (y), with feature selection applied only to the input features.

```
x = df.iloc[:, 0:-1]
y = df.iloc[:, -1]
```

Recursive Feature Elimination (RFE) was then applied using a **Decision Tree** as the base estimator to select the **10** most significant features out of the original 42 input features. These selected features were used consistently across all four classifiers, and the dataset was split into training and testing sets using **Holdout set** with a **70/30 ratio** with **random_state = 42** which produced the best performance in our project.

```
clf = tree.DecisionTreeClassifier()
rfe = RFE(clf, n_features_to_select= 10)
fit = rfe.fit(x,y)

x = x.iloc[:, fit.support_]

x_train , x_test , y_train , y_test = train_test_split(x , y ,
test_size = 0.3 , random_state = 42)
```

Most significant features chosen by RFE were the following:

Feature	Description
sbytes	Source to destination transaction bytes
dbytes	Destination to source transaction bytes
sttl	Source to destination time to live value
dinpkt	Destination interpacket arrival time (mSec)
sjit	Source jitter (mSec)
stcpb	Source TCP base sequence number
dtcpb	Destination TCP base sequence number
tcprtt	TCP connection setup round-trip time, the sum of 'synack' and 'ackdat'
smean	Mean source packet size
ct_srv_dst	Connection count with same service and dst

Outlier removal was deliberately not applied in this study. In network intrusion detection, unusually large or small feature values (e.g., byte counts, packet counts, or durations) often reflect genuine attack behaviour rather than data errors, since different attack types are statistically distinct by nature. A standard IQR-based approach could flag these legitimate patterns as outliers and remove them, disproportionately affecting rare classes like Backdoor and Worms and worsening the existing class imbalance. For this reason, no outlier-based filtering was performed beyond the cleaning steps described above.

V. MODEL DESIGN

A. Decision Tree

```
DT_model=tree.DecisionTreeClassifier(random_state=42)
DT_model.fit(x_train,y_train)
DT_predictions= DT_model.predict(x_test)

print("The accuracy is: ',accuracy_score(y_test,
DT_predictions))

print("The precision is:', precision_score(y_test,
DT_predictions, average='weighted'))
print("The recall is:', recall_score(y_test, DT_predictions,
average='weighted'))
print("The F1 score is:', f1_score(y_test, DT_predictions,
average='weighted'))
```

For implementing the Decision Tree model, the 'DecisionTreeClassifier' class from the **scikit-learn** library was used. Upon splitting the dataset into training and testing datasets, the model was created using 'random_state=2' to ensure reproducibility. Then, it was trained using the 'fit()' method where 'x_train' was the feature array while 'y_train' was the label array of attacks.

Subsequently, the trained model was used to predict the classes in the test set using the 'predict()' method, which produced the predictions stored in the variable 'DT_predictions'. The performance of the model was measured using four parameters: **accuracy, precision, recall, and F1-score**. Precision, recall, and F1-score used weighted averaging because the problem is that of multiclass classification.

B. Random Forest

```
rf_model = RandomForestClassifier(n_estimators=111,
max_depth=7, random_state=14)

rf_model.fit(x_train,y_train)
rf_predictions = rf_model.predict(x_test)

print("The accuracy is:", accuracy_score(y_test,
rf_predictions))
print("The precision is:", precision_score(y_test,
rf_predictions,average='weighted'))
print("The recall is:",recall_score(y_test,
rf_predictions,average='weighted'))
print("The f1 score is:", f1_score(y_test,
rf_predictions,average='weighted'))
```

The Random Forest model was implemented using the **RandomForestClassifier** class from the scikit-learn library. The dataset was first divided into training and testing sets, where **70%** of the data was used for training and **30%** for testing.

The main parameters used were **n_estimators**, **max_depth**, and **random_state**. The number of decision trees (**n_estimators**) was set to **111**, meaning that the model builds 111 decision trees and combines their predictions using majority voting. The maximum depth (**max_depth**) was set to **7** to limit the size of each tree and reduce the possibility of overfitting. The **random_state** parameter was set to **14** to ensure that the same results are obtained each time the model is executed.

Most of the model parameters were left at their default values because the Random Forest classifier already achieved high accuracy and good evaluation metrics without requiring extensive hyperparameter tuning. This indicates that the default settings were sufficient for this dataset.

C. XGBoost

```
XGB_model = XGBClassifier(objective='multi:softmax',
num_class=9, tree_method='hist',
eval_metric='mlogloss', n_estimators=200,
max_depth=6,
learning_rate=0.1,
subsample = 0.8,
random_state=42)
```

```
XGB_model.fit(x_train, y_train)
XGB_predictions = XGB_model.predict(x_test)
```

```
print("The accuracy is:", accuracy_score(y_test,
XGB_predictions))
print("The precision is:", precision_score(y_test,
XGB_predictions,average='weighted'))
print("The recall is:",recall_score(y_test,
XGB_predictions,average='weighted'))
print("The f1 score is:", f1_score(y_test,
XGB_predictions,average='weighted'))
```

The XGBoost model was implemented using the **XGBoost library** and configured through the **XGBClassifier** class which allows different parameters to be configured to control the boosting process, tree complexity, and training behaviour.

The main parameters considered for the XGBoost model include **objective**, **num_class**, **n_estimators**, **max_depth**, **learning_rate**, **subsample**, **tree_method**, and **random_state**. The objective parameter defines the loss function used by the model, **multi:softmax** and **multi:softprob** are used for multiclass classification. After testing both, **We chose multi:softmax** because it was faster and gave us the same results as softprob.

As for **num_class** parameter, it simply specifies the number of output categories in our dataset, which was **9** categories total and for **n_estimators** which is the number of trees built sequentially, We tried 100 and 200, 200 gave us better results and another trial with 300 made us settle on **n_estimators =200** since there wasn't a change in the results. The **max_depth** parameter controls the maximum depth of each decision tree, where deeper trees capture more complex patterns but may lead to overfitting. We went with the default value which is **6**. A **learning rate of 0.1** was selected to provide a balance between learning speed and model accuracy while reducing the risk of overfitting.

The **subsample** parameter specifies the percentage of training samples used for each tree; our model has this set to **0.8** aligning with best practices. The **tree_method** parameter controls the algorithm used for constructing trees, with **hist** providing efficient histogram-based tree construction for large datasets. **eval_metric='mlogloss'** was used to monitor the model's performance during training, as it is the standard evaluation metric for multiclass classification. Lastly, we used **random_state = 42** for reproducibility which is consistent throughout this paper.

This model was evaluated using accuracy, precision, recall and f1 score metrics to have a solid ground of comparison between different models.

D. MLP

```
scaler = StandardScaler()
```

```
x_train = scaler.fit_transform(x_train)
x_test = scaler.transform(x_test)
```

```
model = MLPClassifier(solver="adam", alpha= 0.001 ,
hidden_layer_sizes= (20) , activation="relu" ,
max_iter=500 , tol = 0.0001)
model.fit(x_train , y_train)
```

```
MLP_predictions = model.predict(x_test)
```

```
print("The accuracy is:", accuracy_score(y_test,
MLP_predictions))
print("The precision is:", precision_score(y_test,
MLP_predictions,average='weighted'))
print("The recall is:",recall_score(y_test,
MLP_predictions,average='weighted'))
print("The f1 score is:", f1_score(y_test,
MLP_predictions,average='weighted'))
```

The MLP model was implemented using **the MLPClassifier** class from the scikit-learn library. The dataset was first standardized using **StandardScaler** because MLP performs better when all features are on the same scale. Otherwise, features with larger values can have a greater influence during training, leading to an imbalanced learning process. Which becomes a must since the dataset features have different value ranges, where features such as **sbytes** ,**dbytes** and **stepb** contain much larger values than features such as **teprtt**, **sjit**

and dinpkt, causing larger-scale features to dominate the training process.

The main parameters used were **solver**, **activation**, **hidden_layer_sizes**, **alpha**, **max_iter**, and **tol**. The **Adam** optimizer was selected because it produced better results than SGD and LBFGS, while **ReLU** was used as the hidden-layer activation function since it achieved evaluation metrics above 90%. Different network architectures with one and two hidden layers and hidden layer sizes of (10), (15), (20), and (100) neurons were tested since there are 10 features/inputs. The architecture with one hidden layer containing 20 neurons (**20**) was selected because increasing the number of hidden layers or neurons did not improve the evaluation metrics. The alpha value was set to **0.001** to prevent the model from overfitting the training data, max_iter was set to **500** because the model did not converge with fewer iterations, and **tol** = **0.0001** was used to terminate training when the model converged with an acceptably small error.

Since this is a multiclass classification problem, the **Softmax** activation function is **automatically** applied in the output layer by **MLPClassifier**, so it does not need to be specified manually. After training, the model was evaluated using accuracy, precision, recall, and F1-score, with weighted averaging since it is a multiclass problem.

VI. RESULTS AND DISCUSSION

The performance of each model was evaluated using accuracy, weighted precision, weighted recall and weighted F1 score. These metrics measure the correctness and the classification quality of each model. In addition to the time taken to train the models was recorded to compare computational efficiency. The results obtained are summarized below:

Model	Accuracy	Precision	Recall	F1-Score	Time
Decision Tree	0.9269	0.9281	0.9269	0.9275	0s
Random Forest	0.9260	0.9242	0.9260	0.9085	10s
XGBoost	0.9439	0.9405	0.9439	0.9397	27s
MLP	0.9272	0.9230	0.9272	0.9171	43s

The results show that **all four** models achieved **over 90%** for all evaluation metrics, indicating that each model performed well on the multiclass intrusion detection dataset. Although the differences between the models are relatively small, **XGBoost produced the best overall performance**.

XGBoost being the best model which gave for accuracy, precision, recall and F1 score 94% approximately in 27 seconds. The results shows that XGBoost is highly effective for multiclass classification.

Decision Tree being the second-best model which gave for accuracy, precision, recall and F1 score 93% approximately in 0 seconds. The results show that the Decision Tree provides high classification performance while being the fastest model, making it suitable when computational time is important.

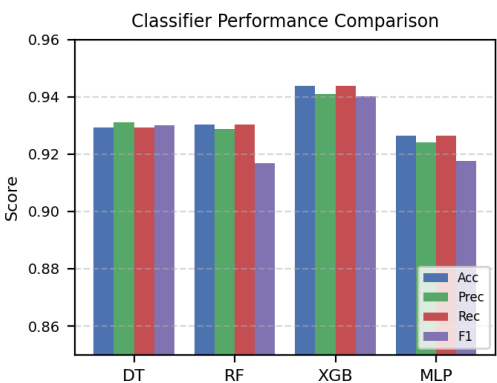
Random Forest being the third model, which gave 92% approximately for accuracy, precision and recall, and 91% for the F1 score in 10 seconds. The results show that Random Forest achieved performance close to the Decision Tree but required a longer computation time.

MLP being the fourth model which gave 92% approximately for accuracy and recall, 92% for precision, and 91% for the F1 score in 43 seconds. The results show that MLP achieved competitive classification performance but required the longest computation time among the four models due to the iterative training process of neural networks.

VVI. CONCLUSION

This paper investigated the use of **Recursive Feature Elimination** for feature selection in multiclass network intrusion detection and compared the performance of four classifiers - **Decision Tree, Random Forest, XGBoost, and MLP** - on the UNSW-NB15 dataset.

The Data was cleaned and encoded with the best techniques since AI models are sensitive to the data. Feature Selection using **RFE** was minimized to the **10** most features that correlated to the output. All four models were trained using the same data set to perform a consistent comparison between the models. XGBoost had the best results with highest accuracy even though Random Forest was expected to have the highest. Decision Tree and Random Forest had close competition, while MLP achieved the lowest performance. Overall, all models produced high results, and it shows how RFE with only 10 features have significantly decreased the size of the data for better processing.



Future work could extend this study by applying explicit class-imbalance handling techniques to improve minority-class detection, experimenting on different values for random forest instead of default values, and evaluating the generalizability of these findings on additional intrusion detection benchmark datasets beyond UNSW-NB15.